

Color Block Volume Measurement

Color Block Volume Measurement

1. Course Content
2. Program Startup
3. Source Code Analysis

1. Course Content

- Run example programs, click on points in the image with the mouse, press spacebar to determine the distance of that point, which serves as the standard distance for volume measurement
- Afterwards, frame select the color block, the program will recognize the shape of the selected color block, obtain relevant data of the color block, and calculate the volume of the color block.

2. Program Startup

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

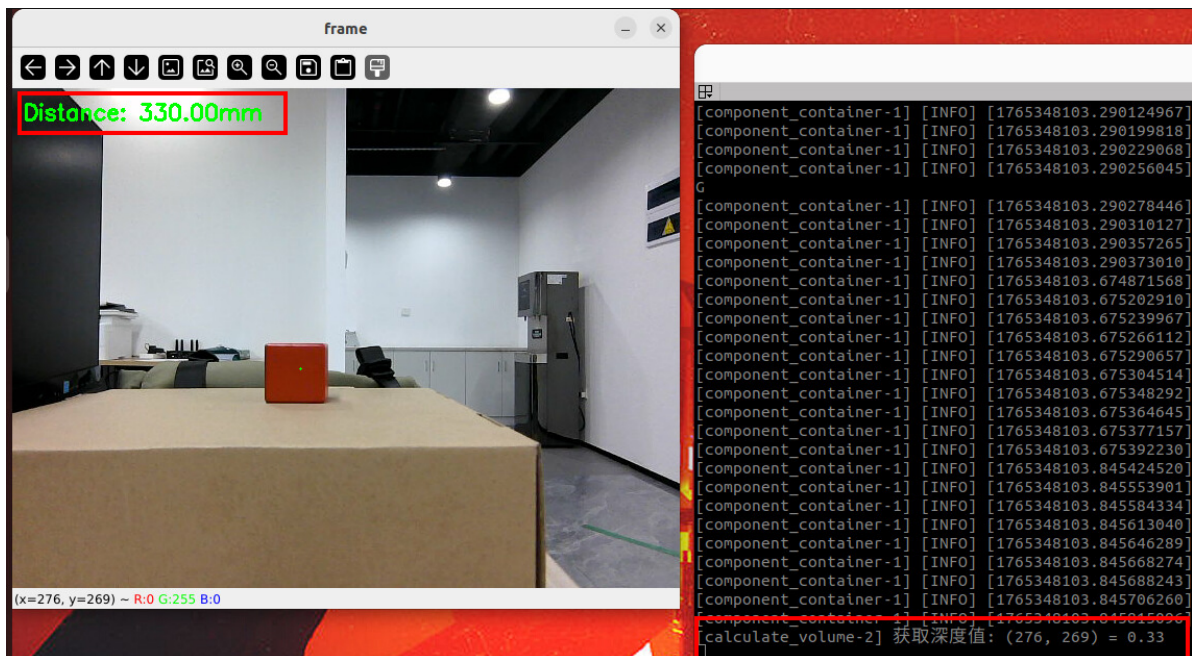
- Run command in terminal:

```
ros2 launch m3_bringup depthcam_demo.launch.py demo:=calculate_volume
```

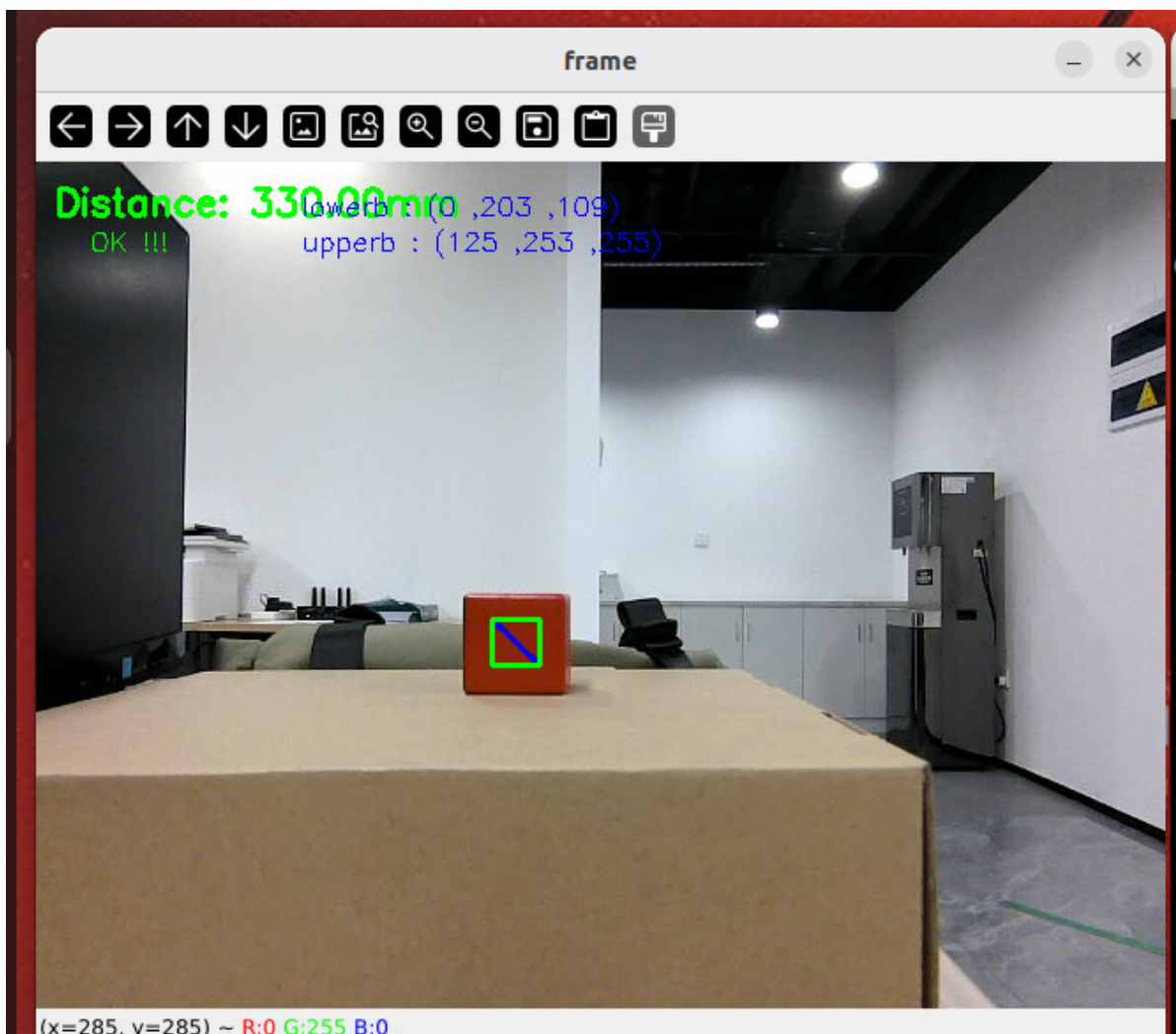
[!WARNING]

Note: The position of the color block needs to be greater than the blind area of the depth camera, recommended to be greater than 0.3 meters

- After the program runs, click the mouse on the calibration distance of the object to be calibrated, press spacebar to calibrate. The terminal will print the calibrated coordinates and distance

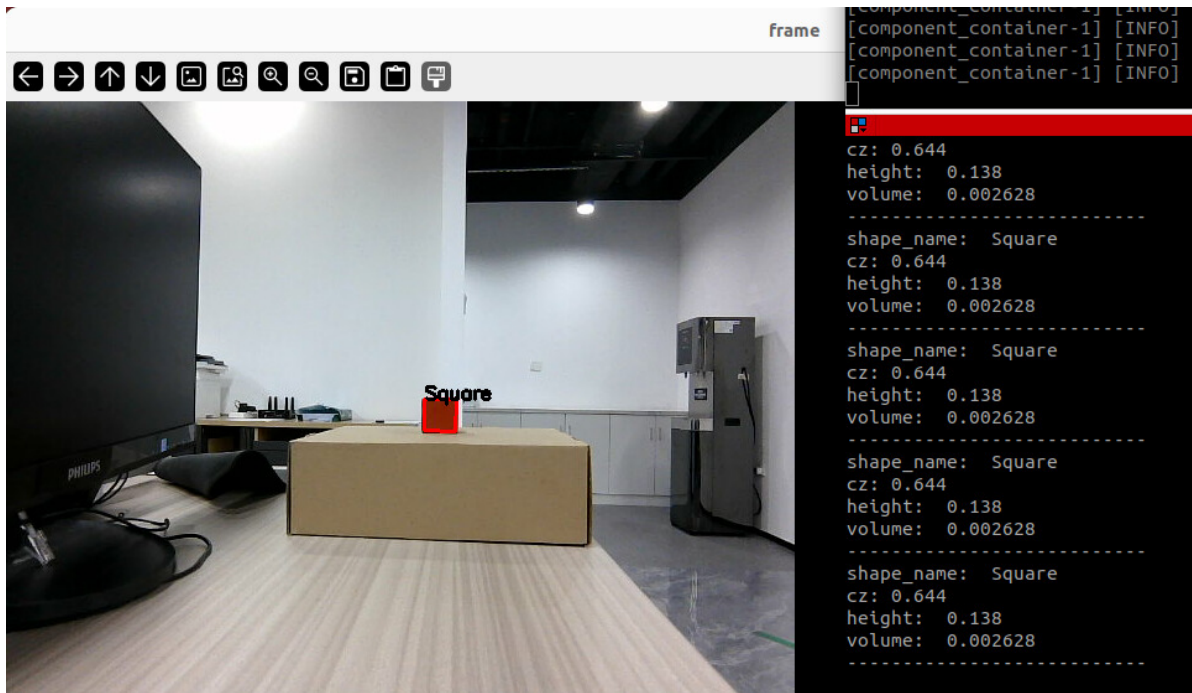


- Hold down the left mouse button without releasing, use the mouse to select an area within the color block to obtain HSV values.



- The program filters out other colors through HSV values, only retaining colors within the HSV value range. The program will recognize the shape of the color block, calculate data of the color block based on its center point, and calculate the volume of the color block based on

the recognized color block shape. (The volume may not be very accurate, this is an estimation)



3. Source Code Analysis

- Source code `calculate_volume.py` path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/depthcamera/calculate_volume.py
```

The main functions of the program are as follows:

- Open camera and get images;
- Get keyboard and mouse events for switching modes and color picking;
- Process images to calculate color block volume based on depth distance information.
- Adjustable HSV color values

Some core code is as follows,

```
def ComputeVolume(self,color_frame,depth_frame):
    # Convert color images to OpenCV format
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame,'bgr8')
    result_image = np.copy(rgb_image)

    # Convert depth image and resize
    depth_image = self.depth_bridge.imgmsg_to_cv2(depth_frame, encoding[1])
    frame = cv.resize(depth_image, (640, 480))
    depth_image_info = frame.astype(np.float32)

    # Process the color image to get the object outline (specifically implemented
    in self.process)
    action = cv.waitKey(10) & 0xFF
    result_image = cv.resize(result_image, (640, 480))
    result_frame, binary= self.process(result_image,action)
```

```

if self.color.shape_cx!=0 and self.color.shape_cy!=0:
    # Rectangle/cylinder processing
    if self.color.shape_name == "Rectangle" or self.color.shape_name ==
"Cylinder":
        # Get the coordinates of three corner points
        x1, y1 = self.get_corner[0], self.get_corner[1]
        x1 = min(max(0, x1), 640 - 1)
        y1 = min(max(0, y1), 480 - 1)
        x1, y1 = int(round(x1)), int(round(y1))

        z1 = depth_image_info[y1,x1]/1000

        # Get the real position of point 1 in the camera coordinate system
        if z1 != 0:
            c1_pose_T = self.get_pos(x1,y1,z1)
        else:
            self.error = True

        # The same process is done for points 2 and 3
        x2, y2 = self.get_corner[2], self.get_corner[3]
        z2 = depth_image_info[y2,x2]/1000

        x3, y3 = self.get_corner[4], self.get_corner[5]
        z3 = depth_image_info[y3,x3]/1000

        # Get the center point depth and calculate the height
        cx, cy = self.color.shape_cx, self.color.shape_cy
        cz = depth_image_info[cy,cx]/1000
        if cz != 0:
            self.center_pose_T = self.get_pos(cx,cy,cz)
            self.height = self.center_pose_T[2]*100 # Convert height to
centimeters

        # If there are no errors, calculate the volume
        if not self.error:
            # Calculate the distance from the point to the center (convert to
centimeters)
            point1 = np.array([c1_pose_T[0], c1_pose_T[1], c1_pose_T[2]])

            c_point = np.array([self.center_pose_T[0],
self.center_pose_T[1], self.center_pose_T[2]])

            r = np.linalg.norm(point1 - c_point)*100

            # Calculate side-to-side distance (convert to centimeters)
            distance1 = np.linalg.norm(point1 - point3)*100
            distance2 = np.linalg.norm(point3 - point2)*100

            # Calculate volume from shape
            if self.color.shape_name == "Rectangle":
                volume = distance1 * distance2 * self.height

            if self.color.shape_name == "Cylinder":
                volume = math.pi * r * r * self.height #  $\pi r^2 h$ 
# Cube processing

```

```
if self.color.shape_name == "Square":
    # Get the center point coordinates
    cx, cy = self.color.shape_cx, self.color.shape_cy
    cx, cy = int(round(cx)), int(round(cy))

    # Get the center point depth
    cz = depth_image_info[cy,cx]/1000

    if cz != 0:
        self.center_pose_T = self.get_pos(cx,cy,cz)
        self.height = self.center_pose_T[2]*100 # Cube side length

        # Volume of a cube (side length³)
        volume = self.height * self.height * self.height
```